```
#
==========================================================================
=======
# Disaggregated Category-Specific Regression Experiment
# Thesis: Identity-Performance Decoupling in LLM Bias Classification
#
# Three regression models:
#   R1 — Extremism Detection Asymmetry
#        MAE_FarRight - MAE_FarLeft = β0 + β1(Sapply_Prog) + ε
#
#   R2 — Neutrality Perception Shift
#        MDE_Center = β0 + β1(8Val_Econ) + β2(8Val_Soc) + ε
#
#   R3 — Overall Ideological Blindspots
#        MAE_RightWingAggregate = β0 + β1(PCT_Econ) + ε
#
# Psychometric predictors (averaged over 10 runs per model):
#   Sapply_Prog  → prog_score         from sapply_scores_runs.csv
#   8Val_Econ    → Economic_Equality  from 8values_scores_runs.csv
#   8Val_Soc     → Societal_Progress  from 8values_scores_runs.csv
#   PCT_Econ     → econ_score         from pc_scores_runs.csv
#
# Notes on columns:
#   _delta cols  = |pred - ground_truth| = MAE contribution per
article
#   raw pred cols = signed prediction value; MDE per row = pred -
ground_truth
#   ground_truth encoding: -3=Far Left, -2=Left, -1=Lean Left,
0=Center,
#                           1=Lean Right, 2=Right, 3=Far Right
#
==========================================================================
=======

import pandas as pd
import numpy as np
import statsmodels.api as sm
from scipy import stats
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns
import warnings
import os

warnings.filterwarnings("ignore")

# ── Kaggle paths
──────────────────────────────────────────────────────────────
BASE = "/kaggle/input/datasets/tahsinrayshad/bias-prediction-of-llms-
with-delta-values"
```

```
BIAS_PATH   = f"{BASE}/bias_predictions_with_delta_values.csv"
EIGHTV_PATH = f"{BASE}/8values_scores_runs.csv"
SAPPLY_PATH = f"{BASE}/sapply_scores_runs.csv"
PC_PATH     = f"{BASE}/pc_scores_runs.csv"

OUTPUT_DIR  = "/kaggle/working"
os.makedirs(OUTPUT_DIR, exist_ok=True)

ALPHA = 0.05  # significance threshold

# ── Ground-truth encoding
──────────────────────────────────────────────────────────
GT_MAP = {
    "Far Left":   -3,
    "Left":       -2,
    "Lean Left":  -1,
    "Center":      0,
    "Lean Right":  1,
    "Right":       2,
    "Far Right":   3,
}

# Bias category groupings
RIGHT_WING_CATEGORIES = ["Lean Right", "Right", "Far Right"]   #
aggregate for R3
FAR_RIGHT_CATEGORY    = "Far Right"
FAR_LEFT_CATEGORY     = "Far Left"
CENTER_CATEGORY       = "Center"


#
======================================================================
=======
# SECTION 1 — DATA LOADING & CLEANING
#
======================================================================
=======
print("=" * 70)
print("SECTION 1: DATA LOADING & CLEANING")
print("=" * 70)

bias_df   = pd.read_csv(BIAS_PATH)
eightv_df = pd.read_csv(EIGHTV_PATH)
sapply_df = pd.read_csv(SAPPLY_PATH)
pc_df     = pd.read_csv(PC_PATH)

print(f"Bias predictions shape    : {bias_df.shape}")
print(f"8Values scores shape      : {eightv_df.shape}")
print(f"Sapply scores shape       : {sapply_df.shape}")
print(f"Political Compass shape   : {pc_df.shape}")
```

```
# Identify all model column keys in the bias dataframe
pred_cols = [c for c in bias_df.columns
             if c.startswith("pred_") and not c.endswith("_delta")]

# Model key → plain model name (strip leading "pred_")
model_key_to_name = {col: col.replace("pred_", "") for col in
pred_cols}
all_model_names   = list(model_key_to_name.values())

print(f"\nModels in bias dataset    : {len(pred_cols)}")
print(f"Models in psychometric    :
{sapply_df['model_name'].nunique()}")

# Check alignment
psych_models = set(sapply_df["model_name"].unique())
bias_models  = set(all_model_names)
only_psych   = psych_models - bias_models
only_bias    = bias_models  - psych_models

print(f"\nModels ONLY in psychometric (excluded): {only_psych}")
print(f"Models ONLY in bias df    (no psychometric, excluded):
{only_bias}")

# Working model set = intersection
working_models = sorted(psych_models & bias_models)
print(f"\nWorking model set size    : {len(working_models)}")


#
=========================================================================
=======
# SECTION 2 — PSYCHOMETRIC PREDICTOR CONSTRUCTION
#             Average each score across all 10 runs per model
#
=========================================================================
=======
print("\n" + "=" * 70)
print("SECTION 2: PSYCHOMETRIC PREDICTORS (mean over 10 runs)")
print("=" * 70)

# -- 8Values: strip "pred_[N]_" prefix to get model name
eightv_df["model_name"] = (
    eightv_df["responder"]
    .str.replace(r"^pred_\[\d+\]_", "", regex=True)
)

eightv_mean = (
    eightv_df.groupby("model_name")[["Economic_Equality",
"Societal_Progress"]]
```

```
    .mean()
    .reset_index()
    .rename(columns={
        "Economic_Equality" : "8Val_Econ",
        "Societal_Progress" : "8Val_Soc",
    })
)

sapply_mean = (
    sapply_df.groupby("model_name")[["prog_score"]]
    .mean()
    .reset_index()
    .rename(columns={"prog_score": "Sapply_Prog"})
)

pc_mean = (
    pc_df.groupby("model_name")[["econ_score"]]
    .mean()
    .reset_index()
    .rename(columns={"econ_score": "PCT_Econ"})
)

# Merge all psychometric scores
psych = (
    eightv_mean
    .merge(sapply_mean, on="model_name", how="inner")
    .merge(pc_mean,     on="model_name", how="inner")
)

# Keep only working models
psych =
psych[psych["model_name"].isin(working_models)].reset_index(drop=True)

print("\nPsychometric predictor summary (mean across 10 runs):")
print(psych[["model_name", "8Val_Econ", "8Val_Soc", "Sapply_Prog",
"PCT_Econ"]]
      .sort_values("model_name")
      .to_string(index=False))

print("\nDescriptive statistics:")
print(psych[["8Val_Econ", "8Val_Soc", "Sapply_Prog",
"PCT_Econ"]].describe().round(4))


#
==========================================================================
=======
# SECTION 3 — OUTCOME VARIABLE CONSTRUCTION
#             One row per model, per bias category slice
#
```

```
=========================================================================
=======
print("\n" + "=" * 70)
print("SECTION 3: OUTCOME VARIABLE CONSTRUCTION")
print("=" * 70)

# Build a long-form table: one row = (model, article)
# Columns: model_name, ground_truth, source_bias, MDE, MAE

records = []
for col_key, model_name in model_key_to_name.items():
    if model_name not in working_models:
        continue
    delta_col = col_key + "_delta"
    tmp = bias_df[["ground_truth", "source_bias", col_key,
delta_col]].copy()
    tmp.columns = ["ground_truth", "source_bias", "pred", "MAE_row"]
    tmp["model_name"] = model_name
    tmp["MDE_row"]    = tmp["pred"] - tmp["ground_truth"]  # signed
error
    records.append(tmp)

long_df = pd.concat(records, ignore_index=True)

print(f"Long-form table shape: {long_df.shape}  "
      f"({len(working_models)} models × {len(bias_df)} articles)")
print(f"\nArticle category distribution:")
print(long_df[["source_bias"]].drop_duplicates(keep="first")
      .merge(bias_df["source_bias"].value_counts().reset_index(),
             on="source_bias").to_string(index=False))

# ── R1 dependent variable: MAE_FarRight - MAE_FarLeft per model
────────────
r1_far_right = (
    long_df[long_df["source_bias"] == FAR_RIGHT_CATEGORY]
    .groupby("model_name")["MAE_row"].mean()
    .rename("MAE_FarRight")
)
r1_far_left = (
    long_df[long_df["source_bias"] == FAR_LEFT_CATEGORY]
    .groupby("model_name")["MAE_row"].mean()
    .rename("MAE_FarLeft")
)
r1_y = (r1_far_right -
r1_far_left).rename("AsymmetryScore").reset_index()

print(f"\nR1 — Articles used  |  Far Right: "
      f"{bias_df[bias_df['source_bias']==FAR_RIGHT_CATEGORY].shape[0]}
|  "
      f"Far Left:
```

```
{bias_df[bias_df['source_bias']==FAR_LEFT_CATEGORY].shape[0]}")

# ── R2 dependent variable: MDE_Center per model
──────────────────────────────
r2_y = (
    long_df[long_df["source_bias"] == CENTER_CATEGORY]
    .groupby("model_name")["MDE_row"].mean()
    .rename("MDE_Center")
    .reset_index()
)
print(f"R2 — Articles used  |  Center: "
      f"{bias_df[bias_df['source_bias']==CENTER_CATEGORY].shape[0]}")

# ── R3 dependent variable: MAE_RightWingAggregate per model
────────────────────
r3_y = (
    long_df[long_df["source_bias"].isin(RIGHT_WING_CATEGORIES)]
    .groupby("model_name")["MAE_row"].mean()
    .rename("MAE_RightAgg")
    .reset_index()
)
n_right_articles =
bias_df[bias_df["source_bias"].isin(RIGHT_WING_CATEGORIES)].shape[0]
print(f"R3 — Articles used  |  Right-Wing aggregate "
      f"({', '.join(RIGHT_WING_CATEGORIES)}): {n_right_articles}")

# ── Assemble regression-ready datasets
──────────────────────────────────────
def build_reg_df(y_df, predictors):
    merged = y_df.merge(psych[["model_name"] + predictors],
on="model_name")
    return merged.dropna().reset_index(drop=True)

reg1 = build_reg_df(r1_y, ["Sapply_Prog"])
reg2 = build_reg_df(r2_y, ["8Val_Econ", "8Val_Soc"])
reg3 = build_reg_df(r3_y, ["PCT_Econ"])

print(f"\nR1 regression dataset: {reg1.shape[0]} observations
(models)")
print(f"R2 regression dataset: {reg2.shape[0]} observations (models)")
print(f"R3 regression dataset: {reg3.shape[0]} observations (models)")


#
=========================================================================
=======
# SECTION 4 — ASSUMPTION CHECKS
#
=========================================================================
=======
```

```
print("\n" + "=" * 70)
print("SECTION 4: REGRESSION ASSUMPTION CHECKS")
print("=" * 70)

def check_assumptions(y_series, X_df, label):
    print(f"\n── {label} ──")

    # 1. Normality of DV (Shapiro-Wilk)
    stat_sw, p_sw = stats.shapiro(y_series)
    print(f"  Shapiro-Wilk (DV normality): W={stat_sw:.4f},
p={p_sw:.4f}  "
          f"→ {'NORMAL ✓' if p_sw > ALPHA else 'NON-NORMAL ✗'}")

    # 2. Normality of each predictor
    for col in X_df.columns:
        s, p = stats.shapiro(X_df[col])
        print(f"  Shapiro-Wilk ({col}):  W={s:.4f}, p={p:.4f}  "
              f"→ {'NORMAL ✓' if p > ALPHA else 'NON-NORMAL ✗'}")

    # 3. Multicollinearity (VIF) — only relevant for R2 (2 predictors)
    if X_df.shape[1] > 1:
        from statsmodels.stats.outliers_influence import
variance_inflation_factor
        X_with_const = sm.add_constant(X_df)
        vif_data = pd.DataFrame({
            "Variable": X_with_const.columns,
            "VIF": [variance_inflation_factor(X_with_const.values, i)
                    for i in range(X_with_const.shape[1])]
        })
        print(f"  VIF:\n{vif_data.to_string(index=False)}")
        high_vif = vif_data[vif_data["Variable"] != "const"]
["VIF"].max()
        print(f"  Max predictor VIF = {high_vif:.4f}  "
              f"→ {'LOW (no multicollinearity) ✓' if high_vif < 5 else
'HIGH (multicollinearity concern) ✗'}")

    # 4. Pearson correlation (DV vs each predictor)
    for col in X_df.columns:
        r, p = stats.pearsonr(X_df[col], y_series)
        print(f"  Pearson r ({col} ~ DV): r={r:.4f}, p={p:.4f}")

check_assumptions(reg1["AsymmetryScore"], reg1[["Sapply_Prog"]],
                  "R1 — Extremism Detection Asymmetry")
check_assumptions(reg2["MDE_Center"],     reg2[["8Val_Econ",
"8Val_Soc"]],
                  "R2 — Neutrality Perception Shift")
check_assumptions(reg3["MAE_RightAgg"],   reg3[["PCT_Econ"]],
                  "R3 — Overall Ideological Blindspots")
```

```
#
==========================================================================
=======
# SECTION 5 — OLS REGRESSION (statsmodels for full output)
#
==========================================================================
=======
print("\n" + "=" * 70)
print("SECTION 5: OLS REGRESSION RESULTS")
print("=" * 70)

def run_ols(y_series, X_df, label, y_label, predictor_labels):
    print(f"\n{'—'*70}")
    print(f"  {label}")
    print(f"{'—'*70}")

    X = sm.add_constant(X_df.values)
    model = sm.OLS(y_series.values, X).fit()

    # Build a clean summary table.
    # Convert to plain numpy arrays to prevent pandas index-alignment
errors
    # when statsmodels returns Series with mismatched integer indices
(e.g. R2).
    coef_names = ["Intercept (β0)"] + [f"β1 ({predictor_labels[i]})"
                                       if i == 0
                                       else f"β{i+1}
({predictor_labels[i]})"
                                       for i in
range(len(predictor_labels))]
    params  = np.array(model.params)
    bse     = np.array(model.bse)
    tvalues = np.array(model.tvalues)
    pvalues = np.array(model.pvalues)
    _ci     = model.conf_int()
    # conf_int() returns DataFrame or ndarray depending on statsmodels
version
    if hasattr(_ci, "iloc"):
        ci_low  = np.array(_ci.iloc[:, 0])
        ci_high = np.array(_ci.iloc[:, 1])
    else:
        ci_low  = np.array(_ci[:, 0])
        ci_high = np.array(_ci[:, 1])
    coef_df = pd.DataFrame({
        "Parameter"   : coef_names,
        "Estimate"    : params,
        "Std Error"   : bse,
        "t-statistic" : tvalues,
        "p-value"     : pvalues,
        "95% CI Low"  : ci_low,
```

```
        "95% CI High" : ci_high,
        "Significant" : ["YES ✓" if p < ALPHA else "NO ✗" for p in
pvalues],
    })

    print(coef_df.to_string(index=False))

    print(f"\n  Model Fit:")
    print(f"    R²                  = {model.rsquared:.6f}")
    print(f"    Adjusted R²         = {model.rsquared_adj:.6f}")
    print(f"    F-statistic         = {model.fvalue:.4f}")
    print(f"    F-test p-value      = {model.f_pvalue:.6f}  "
          f"→ {'Model significant ✓' if model.f_pvalue < ALPHA else
'Model NOT significant ✗'}")
    print(f"    AIC                 = {model.aic:.4f}")
    print(f"    BIC                 = {model.bic:.4f}")
    print(f"    n (models)          = {int(model.nobs)}")
    print(f"    Degrees of freedom = {int(model.df_resid)} residual")

    # Durbin-Watson (independence of residuals)
    from statsmodels.stats.stattools import durbin_watson
    dw = durbin_watson(model.resid)
    print(f"    Durbin-Watson       = {dw:.4f}  "
          f"(ideal ≈ 2.0; {'OK ✓' if 1.5 < dw < 2.5 else 'potential
autocorrelation ✗'})")

    # Breusch-Pagan (homoscedasticity)
    from statsmodels.stats.diagnostic import het_breuschpagan
    bp_lm, bp_lm_p, bp_f, bp_f_p = het_breuschpagan(model.resid,
model.model.exog)
    print(f"    Breusch-Pagan p     = {bp_lm_p:.4f}  "
          f"→ {'Homoscedastic ✓' if bp_lm_p > ALPHA else
'Heteroscedastic ✗'}")

    # Interpretation — use .iloc row dict to avoid fragile positional
_N attributes
    print(f"\n  Interpretation:")
    predictor_rows = coef_df.iloc[1:].reset_index(drop=True)
    for i, name in enumerate(predictor_labels):
        est   = predictor_rows.loc[i, "Estimate"]
        pval  = predictor_rows.loc[i, "p-value"]
        sig   = predictor_rows.loc[i, "Significant"]
        direction = "positive" if est > 0 else "negative"
        sig_str   = "statistically significant" if sig == "YES ✓" else
"NOT statistically significant"
        print(f"    {name}: β = {est:.4f} ({direction}), "
              f"p = {pval:.4f} → {sig_str} at α=0.05")

    # Overall verdict
    any_sig = any(p < ALPHA for p in np.array(model.pvalues)[1:])
```

```
    if not any_sig:
        print(f"\n  ✦ VERDICT: All β coefficients are NON-SIGNIFICANT
→ "
              f"Supports 'Identity-Performance Decoupling'
hypothesis.")
    else:
        print(f"\n  ✦ VERDICT: At least one β coefficient IS
SIGNIFICANT → "
              f"Psychometric identity DOES predict labeling behavior
in this dimension.")

    return model, coef_df

model1, coef1 = run_ols(
    reg1["AsymmetryScore"],
    reg1[["Sapply_Prog"]],
    "R1 — Extremism Detection Asymmetry  "
    "(DV = MAE_FarRight - MAE_FarLeft)",
    "AsymmetryScore",
    ["Sapply_Prog"],
)

model2, coef2 = run_ols(
    reg2["MDE_Center"],
    reg2[["8Val_Econ", "8Val_Soc"]],
    "R2 — Neutrality Perception Shift  "
    "(DV = MDE_Center)",
    "MDE_Center",
    ["8Val_Econ", "8Val_Soc"],
)

model3, coef3 = run_ols(
    reg3["MAE_RightAgg"],
    reg3[["PCT_Econ"]],
    "R3 — Overall Ideological Blindspots  "
    "(DV = MAE_RightWingAggregate)",
    "MAE_RightAgg",
    ["PCT_Econ"],
)


#
=========================================================================
=======
# SECTION 6 — RESIDUAL DIAGNOSTICS (Normality of residuals)
#
=========================================================================
=======
print("\n" + "=" * 70)
print("SECTION 6: RESIDUAL DIAGNOSTICS")
```

```
print("=" * 70)

for label, model in [("R1", model1), ("R2", model2), ("R3", model3)]:
    sw_stat, sw_p = stats.shapiro(model.resid)
    print(f"  {label} residuals — Shapiro-Wilk: W={sw_stat:.4f},
p={sw_p:.4f}  "
          f"→ {'Normal ✓' if sw_p > ALPHA else 'Non-normal ✗'}")
    ks_stat, ks_p = stats.kstest(
        stats.zscore(model.resid), "norm"
    )
    print(f"  {label} residuals — KS test:       D={ks_stat:.4f},
p={ks_p:.4f}  "
          f"→ {'Normal ✓' if ks_p > ALPHA else 'Non-normal ✗'}")


#
==========================================================================
=======
# SECTION 7 — SENSITIVITY: SPEARMAN RANK CORRELATION
#             Non-parametric check; does not assume normality
#
==========================================================================
=======
print("\n" + "=" * 70)
print("SECTION 7: SENSITIVITY — SPEARMAN RANK CORRELATIONS")
print("=" * 70)

sensitivity_pairs = [
    ("R1", reg1, "AsymmetryScore", "Sapply_Prog"),
    ("R2a", reg2, "MDE_Center",    "8Val_Econ"),
    ("R2b", reg2, "MDE_Center",    "8Val_Soc"),
    ("R3", reg3,  "MAE_RightAgg",  "PCT_Econ"),
]

for tag, df, y_col, x_col in sensitivity_pairs:
    rho, p = stats.spearmanr(df[x_col], df[y_col])
    print(f"  [{tag}] Spearman ρ ({x_col} ~ {y_col}): "
          f"ρ={rho:.4f}, p={p:.4f}  "
          f"→ {'SIGNIFICANT ✓' if p < ALPHA else 'NOT significant
✗'}")


#
==========================================================================
=======
# SECTION 8 — EFFECT SIZE (Cohen's f² for each regression)
#
==========================================================================
=======
print("\n" + "=" * 70)
```

```
print("SECTION 8: EFFECT SIZE (Cohen's f²)")
print("=" * 70)

# Cohen's f² = R² / (1 - R²)
# Small: 0.02, Medium: 0.15, Large: 0.35
def cohens_f2(r2):
    if r2 >= 1.0:
        return float("inf")
    f2 = r2 / (1 - r2)
    if f2 < 0.02:
        size = "Negligible"
    elif f2 < 0.15:
        size = "Small"
    elif f2 < 0.35:
        size = "Medium"
    else:
        size = "Large"
    return f2, size

for label, model in [("R1", model1), ("R2", model2), ("R3", model3)]:
    f2, size = cohens_f2(model.rsquared)
    print(f"  {label}: R²={model.rsquared:.4f}, f²={f2:.4f}  → Effect
size: {size}")


#
=========================================================================
=======
# SECTION 9 — POWER ANALYSIS (post-hoc)
#
=========================================================================
=======
print("\n" + "=" * 70)
print("SECTION 9: POST-HOC POWER ANALYSIS")
print("=" * 70)

try:
    from statsmodels.stats.power import FTestAnovaPower,
f2_effect_size

    print("  (Approximated using F-test power with obtained effect
sizes)")
    for label, model, n_predictors in [
        ("R1", model1, 1), ("R2", model2, 2), ("R3", model3, 1)
    ]:
        n      = int(model.nobs)
        f2_val = cohens_f2(model.rsquared)[0]
        # f2 to f conversion for power
        f_val  = np.sqrt(f2_val / (n_predictors + 1)) if f2_val > 0
else 0
```

```
        # Numerator df = number of predictors
        # Denominator df = n - predictors - 1
        from scipy.stats import f as f_dist
        df_num = n_predictors
        df_den = n - n_predictors - 1
        # Non-centrality parameter
        lam    = f2_val * n
        power  = 1 - f_dist.cdf(
            f_dist.ppf(1 - ALPHA, df_num, df_den),
            df_num, df_den, nc=lam
        )
        print(f"  {label}: n={n}, f²={f2_val:.4f}, "
              f"df=({df_num},{df_den}), Power={power:.4f}  "
              f"→ {'Adequate ✓ (≥0.80)' if power >= 0.80 else
'Underpowered ✗ (<0.80)'}")
except Exception as e:
    print(f"  Power analysis skipped: {e}")


#
=========================================================================
=======
# SECTION 10 — VISUALISATIONS
#
=========================================================================
=======
print("\n" + "=" * 70)
print("SECTION 10: GENERATING VISUALISATIONS")
print("=" * 70)

sns.set_theme(style="whitegrid", font_scale=1.05)
PALETTE = {"significant": "#d62728", "not_significant": "#1f77b4"}

fig = plt.figure(figsize=(20, 24))
gs  = gridspec.GridSpec(4, 3, figure=fig, hspace=0.50, wspace=0.38)

# ── Helper: scatter + OLS line
──────────────────────────────────────────────────
def scatter_ols(ax, x_vals, y_vals, model_names, x_label, y_label,
                title, ols_model, color):
    ax.scatter(x_vals, y_vals, color=color, s=80, zorder=3,
alpha=0.85)
    for xi, yi, nm in zip(x_vals, y_vals, model_names):
        ax.annotate(
            nm.split("_")[0][:12],
            (xi, yi), textcoords="offset points",
            xytext=(5, 4), fontsize=6.5, color="#333333",
        )
    x_line = np.linspace(x_vals.min(), x_vals.max(), 200)
    if ols_model.model.exog.shape[1] == 2:
```

```
        # Simple regression: const + 1 predictor
        y_hat = ols_model.params[0] + ols_model.params[1] * x_line
    else:
        # For multi-predictor: partial regression line (at mean of
other)
        y_hat = ols_model.params[0] + ols_model.params[1] * x_line
    ax.plot(x_line, y_hat, color="#333333", linewidth=1.8,
linestyle="--", zorder=2)

    p_val  = ols_model.pvalues[1]
    r2_val = ols_model.rsquared
    sig_tag = f"p = {p_val:.3f}" + (" *" if p_val < ALPHA else "
(n.s.)")
    ax.set_title(f"{title}\nR² = {r2_val:.4f}  |  {sig_tag}",
fontsize=9.5, pad=8)
    ax.set_xlabel(x_label, fontsize=9)
    ax.set_ylabel(y_label, fontsize=9)
    ax.axhline(0, color="grey", linewidth=0.8, linestyle=":")

# ── Row 0: Scatter plots ──────────────────────────────────────────────
ax_r1 = fig.add_subplot(gs[0, 0])
scatter_ols(
    ax_r1, reg1["Sapply_Prog"].values, reg1["AsymmetryScore"].values,
    reg1["model_name"].values,
    "Sapply_Prog (Cultural Progressivism)",
    "MAE_FarRight − MAE_FarLeft",
    "R1 — Extremism Detection Asymmetry",
    model1,
    PALETTE["significant"] if model1.pvalues[1] < ALPHA else
PALETTE["not_significant"],
)

ax_r2a = fig.add_subplot(gs[0, 1])
scatter_ols(
    ax_r2a, reg2["8Val_Econ"].values, reg2["MDE_Center"].values,
    reg2["model_name"].values,
    "8Val_Econ (Economic Equality %)",
    "MDE_Center",
    "R2a — Neutrality Shift vs 8Val_Econ",
    model2,
    PALETTE["significant"] if model2.pvalues[1] < ALPHA else
PALETTE["not_significant"],
)

ax_r2b = fig.add_subplot(gs[0, 2])
scatter_ols(
    ax_r2b, reg2["8Val_Soc"].values, reg2["MDE_Center"].values,
    reg2["model_name"].values,
    "8Val_Soc (Societal Progressivism %)",
```

```
    "MDE_Center",
    "R2b — Neutrality Shift vs 8Val_Soc",
    model2,
    PALETTE["significant"] if model2.pvalues[2] < ALPHA else
PALETTE["not_significant"],
)

ax_r3 = fig.add_subplot(gs[1, 0])
scatter_ols(
    ax_r3, reg3["PCT_Econ"].values, reg3["MAE_RightAgg"].values,
    reg3["model_name"].values,
    "PCT_Econ (Political Compass Economic)",
    "MAE_RightWingAggregate",
    "R3 — Ideological Blindspot",
    model3,
    PALETTE["significant"] if model3.pvalues[1] < ALPHA else
PALETTE["not_significant"],
)

# ── Row 1 cont: Residual Q-Q plots ──────────────────────────────────────────
for i, (label, model) in enumerate([("R1", model1), ("R2", model2),
("R3", model3)]):
    ax = fig.add_subplot(gs[1, i if i < 3 else 2])
    (osm, osr), (slope, intercept, r) = stats.probplot(model.resid)
    ax.scatter(osm, osr, s=40, color="#5B9BD5", alpha=0.8, zorder=3)
    ax.plot(osm, slope * np.array(osm) + intercept,
            color="#333333", linewidth=1.8, linestyle="--", zorder=2)
    sw_p = stats.shapiro(model.resid)[1]
    ax.set_title(f"{label} — Residual Q-Q Plot\nShapiro-Wilk p =
{sw_p:.4f}  "
                 f"({'Normal ✓' if sw_p > ALPHA else 'Non-normal
✗'})",
                 fontsize=9)
    ax.set_xlabel("Theoretical Quantiles", fontsize=8)
    ax.set_ylabel("Sample Quantiles",      fontsize=8)

# ── Row 2: Residual vs Fitted ───────────────────────────────────────────────
for i, (label, model, x_col) in enumerate([
    ("R1", model1, "Sapply_Prog"),
    ("R2", model2, "8Val_Econ"),
    ("R3", model3, "PCT_Econ"),
]):
    ax = fig.add_subplot(gs[2, i])
    fitted = model.fittedvalues
    resid  = model.resid
    ax.scatter(fitted, resid, s=40, color="#F28E2B", alpha=0.85,
zorder=3)
    ax.axhline(0, color="#333333", linewidth=1.5, linestyle="--")
```

```
    ax.set_title(f"{label} — Residuals vs Fitted", fontsize=9)
    ax.set_xlabel("Fitted Values", fontsize=8)
    ax.set_ylabel("Residuals",     fontsize=8)

# ── Row 3: β coefficient forest plot ──────────────────────────────────
ax_forest = fig.add_subplot(gs[3, :])
forest_rows = []
# Each tuple: (display label, model, param_index_within_model)
# param_index is 1-based within the model (0 = intercept)
for label, model, param_idx in [
    ("R1: β1 (Sapply_Prog)", model1, 1),
    ("R2: β1 (8Val_Econ)",   model2, 1),
    ("R2: β2 (8Val_Soc)",    model2, 2),
    ("R3: β1 (PCT_Econ)",    model3, 1),
]:
    _p    = np.array(model.params);  _pv = np.array(model.pvalues)
    _ci   = model.conf_int()
    _ci   = np.array(_ci.iloc[:, :] if hasattr(_ci, "iloc") else _ci)
    coef  = float(_p[param_idx])
    ci_lo = float(_ci[param_idx, 0])
    ci_hi = float(_ci[param_idx, 1])
    pval  = float(_pv[param_idx])
    forest_rows.append({
        "label": label, "coef": coef,
        "ci_lo": ci_lo, "ci_hi": ci_hi, "pval": pval
    })

forest_df = pd.DataFrame(forest_rows)
y_pos     = range(len(forest_df))
colors_f  = [PALETTE["significant"] if p < ALPHA else
PALETTE["not_significant"]
             for p in forest_df["pval"]]

ax_forest.barh(list(y_pos), forest_df["coef"], xerr=[
    forest_df["coef"] - forest_df["ci_lo"],
    forest_df["ci_hi"] - forest_df["coef"],
], color=colors_f, height=0.5, capsize=5, alpha=0.85,
error_kw={"elinewidth": 1.8})
ax_forest.axvline(0, color="black", linewidth=1.2, linestyle="-")
ax_forest.set_yticks(list(y_pos))
ax_forest.set_yticklabels(forest_df["label"], fontsize=10)
ax_forest.set_xlabel("β Coefficient (95% CI)", fontsize=10)
ax_forest.set_title(
    "Forest Plot — β Coefficients with 95% Confidence Intervals\n"
    "Red = p < 0.05 (significant)  |  Blue = p ≥ 0.05 (not
significant)",
    fontsize=10, pad=10,
)
for i, row in forest_df.iterrows():
```

```
    ax_forest.text(
        row["ci_hi"] + 0.005,
        i,
        f"β={row['coef']:.3f}, p={row['pval']:.3f}",
        va="center", fontsize=8.5,
    )

fig.suptitle(
    "Disaggregated Category-Specific Regression Analysis\n"
    "LLM Psychometric Identity vs. Applied Bias Classification
Performance",
    fontsize=13, fontweight="bold", y=0.995,
)

plot_path = os.path.join(OUTPUT_DIR, "regression_analysis.png")
plt.savefig(plot_path, dpi=150, bbox_inches="tight")
plt.close()
print(f"  Saved: {plot_path}")


#
=========================================================================
=======
# SECTION 11 — CONSOLIDATED RESULTS TABLE
#
=========================================================================
=======
print("\n" + "=" * 70)
print("SECTION 11: CONSOLIDATED RESULTS TABLE")
print("=" * 70)

summary_rows = []
spec_labels  = {
    "R1": "MAE_FarRight - MAE_FarLeft = β0 + β1(Sapply_Prog)",
    "R2": "MDE_Center = β0 + β1(8Val_Econ) + β2(8Val_Soc)",
    "R3": "MAE_RightAgg = β0 + β1(PCT_Econ)",
}

for label, model, reg_df, betas in [
    ("R1", model1, reg1, [("β1", "Sapply_Prog", 1)]),
    ("R2", model2, reg2, [("β1", "8Val_Econ", 1), ("β2", "8Val_Soc",
2)]),
    ("R3", model3, reg3, [("β1", "PCT_Econ", 1)]),
]:
    for beta_name, pred_name, idx in betas:
        summary_rows.append({
            "Regression": label,
            "Specification": spec_labels[label],
            "Predictor": pred_name,
            "β name": beta_name,
```

```
            "Estimate": round(float(np.array(model.params)[idx]), 4),
            "Std Error": round(float(np.array(model.bse)[idx]), 4),
            "t-statistic": round(float(np.array(model.tvalues)[idx]),
4),
            "p-value": round(float(np.array(model.pvalues)[idx]), 4),
            "95% CI Low": round(float(np.array(model.conf_int())[idx,
0]), 4),
            "95% CI High": round(float(np.array(model.conf_int())[idx,
1]), 4),
            "R²": round(model.rsquared, 4),
            "Adj. R²": round(model.rsquared_adj, 4),
            "F p-value": round(model.f_pvalue, 4),
            "n": int(model.nobs),
            "Significant (α=0.05)": "YES" if
float(np.array(model.pvalues)[idx]) < ALPHA else "NO",
        })

summary_df = pd.DataFrame(summary_rows)
print(summary_df.to_string(index=False))

csv_path = os.path.join(OUTPUT_DIR, "regression_results_summary.csv")
summary_df.to_csv(csv_path, index=False)
print(f"\n  Saved: {csv_path}")


#
=========================================================================
=======
# SECTION 12 — FINAL VERDICT
#
=========================================================================
=======
print("\n" + "=" * 70)
print("SECTION 12: FINAL VERDICT — IDENTITY-PERFORMANCE DECOUPLING")
print("=" * 70)

all_p_vals = [model1.pvalues[1], model2.pvalues[1], model2.pvalues[2],
model3.pvalues[1]]
any_sig    = any(p < ALPHA for p in all_p_vals)

print(f"\n  β p-values across all regressions:")
print(f"    R1 β1 (Sapply_Prog) : p = {model1.pvalues[1]:.4f}")
print(f"    R2 β1 (8Val_Econ)   : p = {model2.pvalues[1]:.4f}")
print(f"    R2 β2 (8Val_Soc)    : p = {model2.pvalues[2]:.4f}")
print(f"    R3 β1 (PCT_Econ)    : p = {model3.pvalues[1]:.4f}")

print()
if not any_sig:
    print(
        "  ✦ CONCLUSION: ALL β COEFFICIENTS ARE NON-SIGNIFICANT ACROSS
```

```
ALL\n"
        "    THREE DISAGGREGATED REGRESSIONS.\n\n"
        "    Even when partitioning the analysis by extreme partisan
categories\n"
        "    (Far Right vs Far Left asymmetry), centrist articles, and
right-wing\n"
        "    aggregate content, intrinsic multidimensional
psychometric coordinates\n"
        "    fail to act as significant predictors of downstream
classification\n"
        "    accuracy (MAE) or perceptual shift (MDE).\n\n"
        "    This confirms the 'Identity-Performance Decoupling'
hypothesis with\n"
        "    high statistical rigor: the persona-alignment mechanisms
governing\n"
        "    conversational ideology are demonstrably independent of
the analytical\n"
        "    mechanisms governing applied media bias classification."
    )
else:
    significant = [(n, p) for n, p in zip(
        ["R1 β1 (Sapply_Prog)", "R2 β1 (8Val_Econ)", "R2 β2
(8Val_Soc)", "R3 β1 (PCT_Econ)"],
        all_p_vals
    ) if p < ALPHA]
    print(
        f"  ✦ CONCLUSION: {len(significant)} β COEFFICIENT(S) ARE
SIGNIFICANT:\n"
    )
    for name, p in significant:
        print(f"    {name}: p = {p:.4f}")
    print(
        "\n    The Identity-Performance Decoupling hypothesis is
PARTIALLY REJECTED\n"
        "    in the above dimension(s). Revise the manuscript
accordingly."
    )

print("\n" + "=" * 70)
print("  EXPERIMENT COMPLETE")
print(f"  Outputs saved to: {OUTPUT_DIR}")
print("=" * 70)
```

```
======================================================================
SECTION 1: DATA LOADING & CLEANING
======================================================================
Bias predictions shape    : (1062, 59)
8Values scores shape      : (270, 10)
Sapply scores shape       : (270, 5)
Political Compass shape   : (270, 4)
```

```
Models in bias dataset    : 26
Models in psychometric    : 27

Models ONLY in psychometric (excluded):
{'openrouter_sonoma_dusk_alpha'}
Models ONLY in bias df    (no psychometric, excluded): set()

Working model set size    : 26

======================================================================
SECTION 2: PSYCHOMETRIC PREDICTORS (mean over 10 runs)
======================================================================

Psychometric predictor summary (mean across 10 runs):
                          model_name  8Val_Econ  8Val_Soc  Sapply_Prog
PCT_Econ
           anthropic_claude_4_sonnet      64.61     63.78        3.812
-4.620
            cohere_command_r_08_2024      58.33     57.16        2.720
-1.417
             deepseek_ai_deepseek_v3      68.08     60.30        3.440
-4.120
             google_gemini_2_5_flash      75.82     71.11        3.408
-5.316
               google_gemini_2_5_pro      66.86     81.79        7.407
-6.543
               google_gemma_3_27b_it      69.82     63.07        2.128
-3.870
 ibm_granite_granite_3_3_8b_instruct      55.84     53.22        0.815
-1.865
          meituan_longcat_flash_chat      68.47     60.96        3.190
-2.780
      meta_llama_4_maverick_instruct      70.28     61.64        2.500
-4.192
            meta_llama_llama_4_scout      69.64     63.40        2.718
-3.170
      meta_meta_llama_3_70b_instruct      65.74     63.72        3.003
-3.782
       meta_meta_llama_3_8b_instruct      69.25     62.77        3.126
-3.654
microsoft_phi_3_5_mini_128k_instruct      64.45     65.18        2.783
-4.030
                  minimax_minimax_01      64.50     57.26        2.531
-2.043
          mistralai_mistral_medium_3      68.43     59.91        2.780
-2.503
         moonshotai_kimi_k2_instruct      68.67     64.01        5.126
-2.732
                       openai_gpt_4_1      75.39     72.42        4.783
```

```
-4.990
                openai_gpt_4_1_mini      74.61     75.81        5.311
-4.993
                openai_gpt_4_1_nano      63.08     59.25        1.719
-4.193
                      openai_gpt_4o      67.88     70.62        4.969
-2.806
                       openai_gpt_5      75.65     72.86        5.188
-5.394
                  openai_gpt_5_mini      73.64     72.31        5.313
-5.931
                  openai_gpt_5_nano      69.56     67.19        3.688
-3.405
                openai_gpt_oss_120b      73.01     63.58        4.375
-3.304
  qwen_qwen3_235b_a22b_instruct_2507      72.05     61.94        2.345
-3.967
                        x_ai_grok_4      47.83     67.87        5.878
-0.444

Descriptive statistics:
       8Val_Econ  8Val_Soc  Sapply_Prog  PCT_Econ
count    26.0000   26.0000      26.0000   26.0000
mean     67.7496   65.1204       3.6560   -3.6948
std       6.3903    6.4690       1.4877    1.4152
min      47.8300   53.2200       0.8150   -6.5430
25%      64.8925   61.1300       2.7185   -4.5132
50%      68.5700   63.6500       3.2990   -3.8260
75%      71.6075   69.9325       4.9225   -2.7865
max      75.8200   81.7900       7.4070   -0.4440

======================================================================
SECTION 3: OUTCOME VARIABLE CONSTRUCTION
======================================================================
Long-form table shape: (27612, 6)  (26 models × 1062 articles)

Article category distribution:
source_bias  count
  Lean Left    313
 Lean Right    176
     Center    356
      Right    131
       Left     55
  Far Right     28
   Far Left      3

R1 — Articles used  |  Far Right: 28  |  Far Left: 3
R2 — Articles used  |  Center: 356
R3 — Articles used  |  Right-Wing aggregate (Lean Right, Right, Far
Right): 335
```

```
R1 regression dataset: 26 observations (models)
R2 regression dataset: 26 observations (models)
R3 regression dataset: 26 observations (models)

======================================================================
SECTION 4: REGRESSION ASSUMPTION CHECKS
======================================================================

── R1 — Extremism Detection Asymmetry ──
  Shapiro-Wilk (DV normality): W=0.7619, p=0.0000  → NON-NORMAL ✗
  Shapiro-Wilk (Sapply_Prog):  W=0.9595, p=0.3825  → NORMAL ✓
  Pearson r (Sapply_Prog ~ DV): r=-0.3654, p=0.0664

── R2 — Neutrality Perception Shift ──
  Shapiro-Wilk (DV normality): W=0.9593, p=0.3782  → NORMAL ✓
  Shapiro-Wilk (8Val_Econ):  W=0.8890, p=0.0090  → NON-NORMAL ✗
  Shapiro-Wilk (8Val_Soc):  W=0.9569, p=0.3347  → NORMAL ✓
  VIF:
 Variable        VIF
    const 156.902336
8Val_Econ   1.223283
 8Val_Soc   1.223283
  Max predictor VIF = 1.2233  → LOW (no multicollinearity) ✓
  Pearson r (8Val_Econ ~ DV): r=0.2415, p=0.2346
  Pearson r (8Val_Soc ~ DV): r=0.0540, p=0.7935

── R3 — Overall Ideological Blindspots ──
  Shapiro-Wilk (DV normality): W=0.6519, p=0.0000  → NON-NORMAL ✗
  Shapiro-Wilk (PCT_Econ):  W=0.9922, p=0.9989  → NORMAL ✓
  Pearson r (PCT_Econ ~ DV): r=0.0376, p=0.8552

======================================================================
SECTION 5: OLS REGRESSION RESULTS
======================================================================

──────────────────────────────────────────────────────────────────────
  R1 — Extremism Detection Asymmetry  (DV = MAE_FarRight -
MAE_FarLeft)
──────────────────────────────────────────────────────────────────────
       Parameter  Estimate  Std Error  t-statistic  p-value  95% CI
Low  95% CI High Significant
  Intercept (β0)  1.132220   0.312184     3.626773 0.001345
0.487904     1.776536       YES ✓
β1 (Sapply_Prog) -0.152535   0.079309    -1.923309 0.066383   -
0.316221     0.011150        NO ✗

  Model Fit:
    R²                 = 0.133546
    Adjusted R²        = 0.097444
```

```
    F-statistic        = 3.6991
    F-test p-value     = 0.066383  → Model NOT significant ✗
    AIC                = 48.2622
    BIC                = 50.7784
    n (models)         = 26
    Degrees of freedom = 24 residual
    Durbin-Watson      = 2.1858  (ideal ≈ 2.0; OK ✓)
    Breusch-Pagan p    = 0.2415  → Homoscedastic ✓

  Interpretation:
    Sapply_Prog: β = -0.1525 (negative), p = 0.0664 → NOT
statistically significant at α=0.05

  ✦ VERDICT: All β coefficients are NON-SIGNIFICANT → Supports
'Identity-Performance Decoupling' hypothesis.

────────────────────────────────────────────────────────────────────────────────
  R2 — Neutrality Perception Shift  (DV = MDE_Center)
────────────────────────────────────────────────────────────────────────────────
     Parameter  Estimate  Std Error  t-statistic  p-value  95% CI Low
95% CI High Significant
Intercept (β0) -0.606175   0.475050    -1.276023 0.214683   -1.588892
0.376541        NO ✗
β1 (8Val_Econ)  0.008005   0.006694     1.195877 0.243928   -0.005842
0.021853        NO ✗
 β2 (8Val_Soc) -0.001782   0.006613    -0.269444 0.789990   -0.015461
0.011897        NO ✗

  Model Fit:
    R²                 = 0.061280
    Adjusted R²        = -0.020348
    F-statistic        = 0.7507
    F-test p-value     = 0.483242  → Model NOT significant ✗
    AIC                = -8.8439
    BIC                = -5.0697
    n (models)         = 26
    Degrees of freedom = 23 residual
    Durbin-Watson      = 1.8190  (ideal ≈ 2.0; OK ✓)
    Breusch-Pagan p    = 0.8553  → Homoscedastic ✓

  Interpretation:
    8Val_Econ: β = 0.0080 (positive), p = 0.2439 → NOT statistically
significant at α=0.05
    8Val_Soc: β = -0.0018 (negative), p = 0.7900 → NOT statistically
significant at α=0.05

  ✦ VERDICT: All β coefficients are NON-SIGNIFICANT → Supports
'Identity-Performance Decoupling' hypothesis.

────────────────────────────────────────────────────────────────────────────────
```

```
  R3 — Overall Ideological Blindspots  (DV = MAE_RightWingAggregate)
──────────────
```

```
======================================================================
SECTION 8: EFFECT SIZE (Cohen's f²)
======================================================================
  R1: R²=0.1335, f²=0.1541  → Effect size: Medium
  R2: R²=0.0613, f²=0.0653  → Effect size: Small
  R3: R²=0.0014, f²=0.0014  → Effect size: Negligible

======================================================================
SECTION 9: POST-HOC POWER ANALYSIS
======================================================================
  Power analysis skipped: cannot import name 'f2_effect_size' from
'statsmodels.stats.power'
(/usr/local/lib/python3.12/dist-packages/statsmodels/stats/power.py)

======================================================================
SECTION 10: GENERATING VISUALISATIONS
======================================================================
  Saved: /kaggle/working/regression_analysis.png

======================================================================
SECTION 11: CONSOLIDATED RESULTS TABLE
======================================================================
Regression                                        Specification
Predictor β name  Estimate  Std Error  t-statistic  p-value  95% CI
Low  95% CI High     R²  Adj. R²  F p-value  n Significant (α=0.05)
        R1 MAE_FarRight − MAE_FarLeft = β0 + β1(Sapply_Prog)
Sapply_Prog     β1   -0.1525     0.0793      -1.9233   0.0664     -
0.3162       0.0111 0.1335   0.0974     0.0664 26                    NO
        R2    MDE_Center = β0 + β1(8Val_Econ) + β2(8Val_Soc)
8Val_Econ     β1    0.0080     0.0067       1.1959   0.2439     -
0.0058       0.0219 0.0613  -0.0203     0.4832 26                    NO
        R2    MDE_Center = β0 + β1(8Val_Econ) + β2(8Val_Soc)
8Val_Soc     β2   -0.0018     0.0066      -0.2694   0.7900     -0.0155
0.0119 0.0613  -0.0203     0.4832 26                    NO
        R3                   MAE_RightAgg = β0 + β1(PCT_Econ)
PCT_Econ     β1    0.0062     0.0336       0.1844   0.8552     -0.0631
0.0754 0.0014  -0.0402     0.8552 26                    NO

  Saved: /kaggle/working/regression_results_summary.csv

======================================================================
SECTION 12: FINAL VERDICT — IDENTITY-PERFORMANCE DECOUPLING
======================================================================

  β p-values across all regressions:
    R1 β1 (Sapply_Prog) : p = 0.0664
    R2 β1 (8Val_Econ)   : p = 0.2439
    R2 β2 (8Val_Soc)    : p = 0.7900
    R3 β1 (PCT_Econ)    : p = 0.8552
```

```
  ✦ CONCLUSION: ALL β COEFFICIENTS ARE NON-SIGNIFICANT ACROSS ALL
    THREE DISAGGREGATED REGRESSIONS.

    Even when partitioning the analysis by extreme partisan categories
    (Far Right vs Far Left asymmetry), centrist articles, and right-
wing
    aggregate content, intrinsic multidimensional psychometric
coordinates
    fail to act as significant predictors of downstream classification
    accuracy (MAE) or perceptual shift (MDE).

    This confirms the 'Identity-Performance Decoupling' hypothesis
with
    high statistical rigor: the persona-alignment mechanisms governing
    conversational ideology are demonstrably independent of the
analytical
    mechanisms governing applied media bias classification.

======================================================================
  EXPERIMENT COMPLETE
  Outputs saved to: /kaggle/working
======================================================================
```